



MINISTÈRIO DA EDUCAÇÃO
UNIVERSIDADE FEDERAL DE ITAJUBÁ

Criada pela Lei nº 10.435 – 24/04/2002

Notas sobre o uso de estruturas

- (1) Considere a struct ponto definida abaixo e um vetor de estruturas struct ponto, denominado vet, de tamanho 2

```
struct ponto{  
    int x, y;  
}
```

Na declaração **e somente na declaração**, vet pode ser inicializado das seguintes maneiras:

```
struct ponto vet[2] = {10,20,30,40} ou struct ponto vet[2] = {{10,20},{30,40}}
```

Nas duas formas, teremos os seguintes valores para os campos x e y dos elementos dos vetores:

```
vet[0].x = 10 e vet[0].y = 20
```

```
vet[1].x = 30 e vet[1].y = 40
```

- (2) Em C, é possível redefinir um tipo existente atribuindo um novo nome ao tipo. Isto é feito usando o comando typedef.

Veja o código abaixo. O tipo **int** foi renomeado para **inteiro** e o tipo **float** para **decimal**. Tendo feito esta renomeação, as variáveis dos tipos int e float podem ser declaradas usando os novos identificadores de tipos criados: inteiro e decimal, respectivamente.

```
typedef int inteiro;  
typedef float decimal;  
int main () {  
    inteiro código;  
    decimal preco;  
    printf("\nDigite o código e o preço do produto: ");  
    scanf("%d %f", &código, &preco);  
    return 0;  
}
```

Ao declarar uma estrutura, não basta usar somente o identificador da estrutura, é necessário usar também a palavra chave struct. Por exemplo, ao declarar o vetor vet, foi usado **struct ponto vet[2]**.

Para simplificar esta declaração, podemos usar o typedef para definir um novo nome (identificador) para estrutura:



MINISTÈRIO DA EDUCAÇÃO
UNIVERSIDADE FEDERAL DE ITAJUBÁ
Criada pela Lei nº 10.435 – 24/04/2002

```
struct ponto{
    int x y;
}
typedef struct ponto tponto;
```

A partir da redefinição do tipo **struct ponto** para **tponto**, podemos usar este novo tipo para declarar novas estruturas do tipo struct ponto:

Veja o exemplo abaixo:

```
struct ponto{
    int x, y;
}
typedef struct ponto tponto;
int main(){
    tponto p;
    p.x = 10; p.y=20;
}
Os dois blocos de instruções abaixo são idênticos:
Bloco 1:
struct ponto{
    int x, y;
}
typedef struct ponto tponto;

Bloco 2:
typedef struct ponto{
    int x, y;
} tponto;
```

- (3) Uma estrutura pode ser usada para definir um campo de outra estrutura, por exemplo:

```
struct figura {
    char nome[50];
    tponto posicao;
};
```

Veja que a struct ponto foi usada para definir um campo da struct figura.

Agora, usando o typedef:

```
struct ponto {int, x, y;}
typedef struct ponto tponto;
struct figura {char nome[50];
tponto posicao;
```



MINISTÉRIO DA EDUCAÇÃO UNIVERSIDADE FEDERAL DE ITAJUBÁ

Criada pela Lei nº 10.435 – 24/04/2002

```
};  
typedef struct figura tfigura;
```

Na função principal ou nas demais funções implementadas em um programa, os campos da estrutura `figura` são acessados da seguinte forma:

```
int main(){  
    tfigura f;  
    strcpy(f.nome, "Quadrado");  
    f.posicao.x = 10; //acesso ao campo x do campo posição da figura  
    f.posicao.y = 20; //acesso ao campo y do campo posição da figura  
}
```

(4) Considere o tipo `taluno` criado da seguinte maneira:

```
typedef struct aluno{  
    int codigo;  
    float nota;  
} taluno;
```

Na função `main`, `pa` é um ponteiro para uma variável do tipo `taluno`. Quando temos um ponteiro para uma estrutura, acessamos os campos da estrutura com a seta:

```
int main(void) {  
    taluno a, *pa;  
    pa = &a;  
    pa->codigo = 10; /*pa.codigo, campo código da estrutura a apontada por pa  
    pa->nota = 8;    /* pa.nota, campo nota da estrutura a apontada por pa  
    return 0;  
}
```

(5) Quando é necessário alterar os valores dos campos de uma estrutura em uma função, assim como para os demais tipos de variáveis, temos que passar um ponteiro para essa estrutura (passagem de parâmetro por referência). Veja no exemplo abaixo:

```
void preencheestrutura(taluno *a){  
    printf("Digite o código e a nota do aluno: ");  
    scanf("%d %f", &a->codigo, &a->nota); /* os campos código e nota da estrutura apontada  
    por a é alterado na função preencheestrutura */  
    return;}  
int main(void) {  
    taluno a;  
    preencheestrutura(&a);  
    printf("\nO código e a nota são %d e %f", a.codigo, a.nota);  
    return 0;}
```